

Metric-based defect prediction from class diagram

Batnyam Battulga^{ID}, Lkhamrolom Tsoodol^{ID}, Enkhzol Dovdon^{ID}, Naranchimeg Bold^{ID}*,
Oyun-Erdene Namsrai^{ID}

Department of Information and Computer Sciences, School of Information Technology and Electronics, National University of Mongolia, Mongolia

ARTICLE INFO

Dataset link: <https://gitlab.com/design-metric/metric-based-design-defect-dataset>

Keywords:

Software defect prediction
UML Class diagram
Machine learning
Deep learning
Object oriented design metrics
Reverse-engineering
Software quality
Empirical software engineering

ABSTRACT

A software defect refers to a fault, failure, or error in software. With the rapid development and increasing reliance on software products, it is essential to identify these defects as early and easily as possible, given the efforts and budget invested in their creation and maintenance. In the literature, various approaches such as machine learning (ML) and deep learning (DL), have been proposed and proven effective in detecting defects in source code during the implementation or testing phases of the software development life cycle (SDLC). A promising approach is crucial for predicting defects at earlier stages of the SDLC, particularly during the design phase, with the goal of enhancing software quality while reducing time, effort, and costs. Meanwhile, software metrics provide a quantifiable way to analyze the software, making it easier to identify defects. Many researchers have leveraged these metrics to predict defects using ML and DL methods, achieving state-of-the-art performance. The objective of this paper is to present a novel approach to predict defects in class diagram (i.e., at design stage) using ML and DL with software metrics. Due to a lack of defect datasets extracted from class diagram, firstly, we created a model-based metric dataset using reverse engineering from a code-based dataset. Then, we apply various ML and DL techniques to the newly created dataset to predict defects in classes by classifying them as either defective or clean. The study utilizes a large dataset called the Unified Bug Dataset, which comprises five publicly available sub-datasets. We compare ML and DL models in terms of accuracy, precision, recall, F-measure, AUC and provide a performance comparison against code-based methods. Finally, we conducted a cross-dataset experiment to evaluate the generalizability of our approach.

1. Introduction

In the past few decades, with the increasing requirements of users and the rapid-increasing demand for software, the quality of software has become a critical issue, since these software often highly complicated and failure-prone [1,2]. Finding and fixing software defects, especially in large project software, is very complicated, expensive, and labor-intensive. That is why predicting the defective modules or parts in a software system as early as possible is a very crucial activity, since it leads to a decrease in the total cost of the project and increases the overall project success rate [2]. Because the defects occurring at the early stages of the software development life cycle (SDLC) might propagate to the subsequent phases, further increasing the number of defects [3]. Therefore, early defect prediction is demanded more and considered a key challenging task for the successful software development.

In the literature, a large number of studies [2,4–6] on software defect/bug prediction proposed to obtain defect probability of code areas that potentially contain defects using features of source codes. Numerous prediction models, including machine learning [2,7] and

deep learning techniques [6,8,9], have been developed and demonstrate state-of-the-art performance. Based on the features they use, the models can be mainly divided into two categories: models that use software metrics [10–12] and models that use automatically learned features [13–15] from source codes.

The prediction model performs best if there is enough data to train the model. Therefore, several source code-based datasets [9,16] have been created and used by many researchers to build and validate novel defect prediction models from source code. Recently, Ferenc et al. introduced the Unified bug dataset [17], a comprehensive repository containing code metrics and bug information measured from the source code of a total of 82 open-source programs from five public bug datasets. The following work [12] by the same authors proposed deep learning-based prediction model showing that the static metrics can be meaningful features.

Despite the success of defect or bug prediction from source code, the evaluation of defect prediction from UML class diagram (i.e., defect prediction at the design phase in SDLC) has been sparsely treated so

* Corresponding author.

E-mail address: naranchimeg@num.edu.mn (N. Bold).

<https://doi.org/10.1016/j.array.2025.100438>

Received 29 October 2024; Received in revised form 16 June 2025; Accepted 18 June 2025

Available online 1 July 2025

2590-0056/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

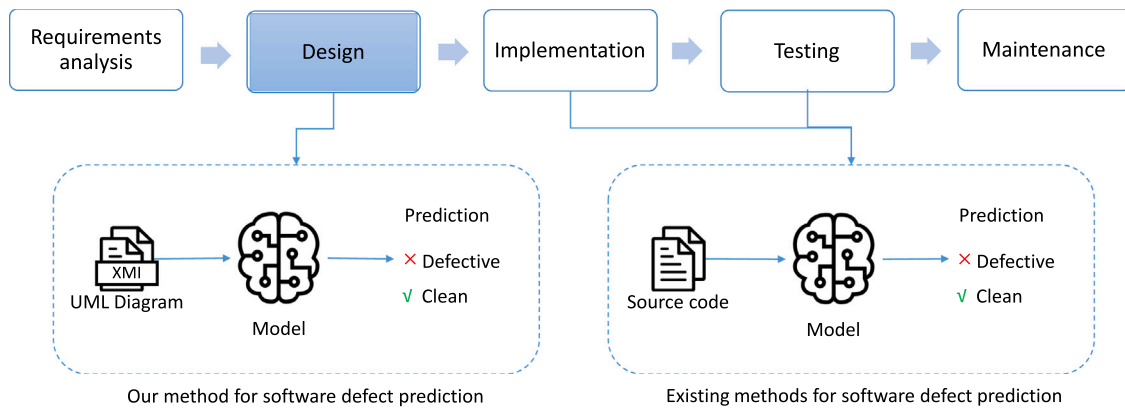


Fig. 1. Advantages of our proposed method over existing software defect prediction approaches.

far. In recent studies [18] in design smells detection, authors created decision tree-based model using software metrics based on the data reverse-engineered from 5 open-source projects. Contemporary, self-supervised learning model [19] has been developed to detect design smells by learning masked keywords in class diagrams. Therefore, a promising approach for predicting defects from class diagram, is essential. Due to the lack of defect datasets extracted from class diagram, our plan is to take the aforementioned defect dataset, use it to create a model-based defect dataset with metrics extracted from class diagram, and build and validate design defect prediction models. Moreover, we compare the performances of ML and DL models trained on the newly created dataset with those trained on the original code-based dataset.

The advantages of our proposed method against existing software defect prediction methods are shown in Fig. 1. As illustrated in Fig. 1, most state-of-the-art methods [2,4,6] in software defect prediction primarily operate on source code and utilize machine learning or deep learning techniques. In contrast, our proposed method analyzes class diagrams instead of source code, demonstrating promising performance in comparison. Therefore, our approach offers three main advantages. First, it enables software defect prediction at an early stage of the Software Development Life Cycle (SDLC), specifically during the design phase. Second, it achieves promising results when compared to existing code-based methods. Third, we have constructed a model-based metric dataset derived from class diagrams, which can serve as a valuable resource for future research. The overview of the proposed method is shown in Fig. 2. Our main contributions are summarized as follows:

- We propose a novel approach for predicting defects in class diagrams using machine learning and deep learning
- We create a model-based defect dataset from a code-based dataset [17] using reverse engineering.
- We empirically show that our approach achieves consistent results compared to code-based approaches.

The rest of the paper organized is as follows. In Section 2, we review related works on software defect prediction. Section 3 describes the proposed method including dataset creation, model training and performance evaluation. Section 4 presents the experiments and results. Section 5 discusses the threads to validity of our study. Finally, we present a summary of our conclusions and discuss future work in Section 6.

2. Related works

Software defect prediction is a field of interest in both academic literature [2,4,6,9] and the software industry [5].

Software defect prediction primarily involves models designed to identify faulty parts of software, with most being developed at the code level. Previous studies towards defect prediction in code level,

mostly composed of extracting features from bug/defect datasets and building classifiers using ML or DL models [2,6,9]. The models can be classified into two types based on their features: those that use software metrics [10–12] and those that employ automatically learned features [13–15].

Many works concentrated on static code metrics as the discriminative features of software defects, including traditional metrics (e.g., size, complexity), object-oriented metrics (i.e., CK metrics such as number of children, coupling between object classes), process-based metrics (e.g., add or deleted lines, the changes between versions), etc. Among them, research works with high accuracy used the multiple linear regression [20–22] and linear regression method [23] during testing, while the random forest algorithm [24] was used to predict class defects during development. They calculated those metrics using various tools on different datasets [16]. Recently, Ferenc et al. [17] introduced a large bug dataset uniting existing public bug dataset and extracted 60 software metrics using OpenStaticAnalyzer toolset and applied deep neural networks to flag suspicious code segments [12]. Furthermore, Goyal [11] proposed a deep learning-based model on the Promise dataset using static software metrics and outperformed the state-of-the-art approaches, showing that the static metrics can be meaningful predictors.

However, to the best of our knowledge, only a few studies have used UML models for design smells detection at the model level. Oyun et al. [25] proposed software fault proneness method using data mining techniques from UML class diagram. They used 4 open source projects and 33 design metrics. Maddeh et al. [18] proposed a decision tree-based model using 15 object-oriented metrics extracted from UML class diagrams to detect design smells. They evaluated the model on five open source projects. Most recent work is proposed by Alazba et al. [19] that learns UML class features by training Transformers [26]. They created UML class dataset using reverse engineering on AP-MART¹ dataset.

Therefore, in this study, we aim to identify defects at the model level by adapting a large bug dataset and utilizing ML and DL techniques.

3. Methodology

This section discusses the proposed method. First, it describes the overall process for creating model-based metrics datasets from source code repositories, which includes generating class diagrams, extracting design metrics from these diagrams, and aligning labels with code-based metrics datasets (Unified bug dataset by Ferenc et al. [17]). Next, the training of ML and DL models to classify a class as defective or clean is described. Later, the evaluation metrics to measure the performance of models are discussed.

¹ <https://www.ptidej.net/tools/designsmells>.

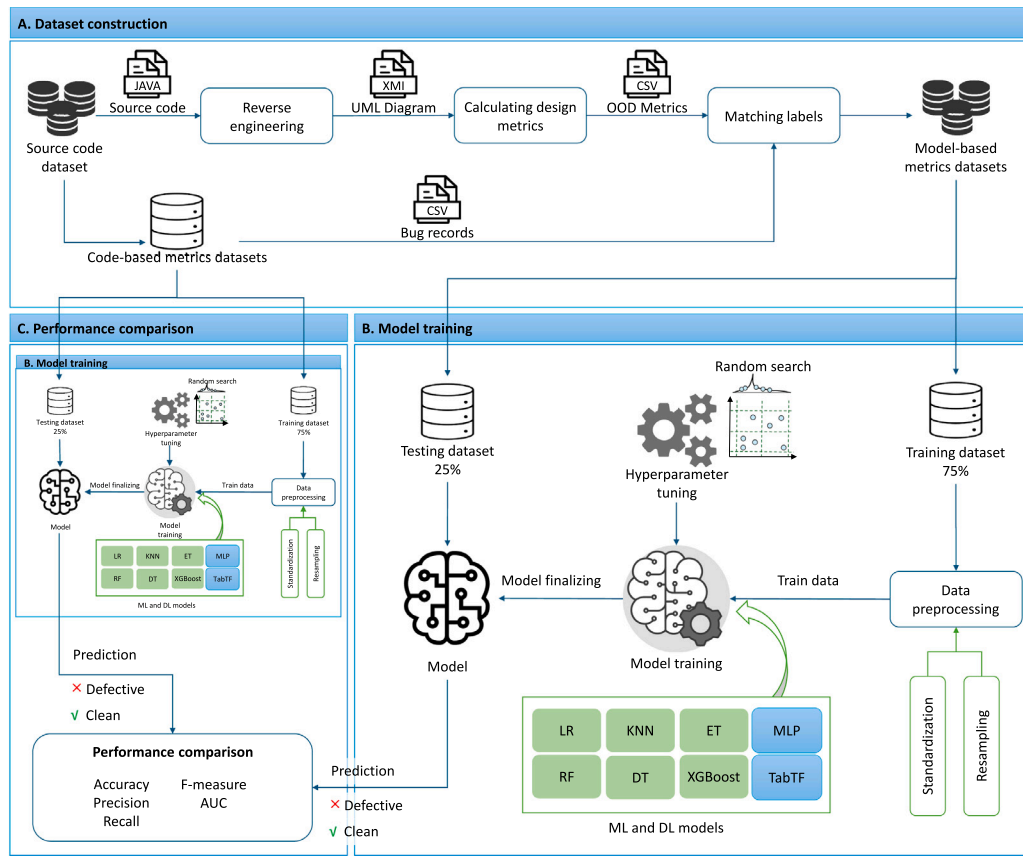


Fig. 2. Overview of the proposed method. A. we illustrate the pipeline of generating software metrics from class diagrams obtained from reverse-engineered source codes. B. we demonstrate training process of ML (green) and DL (blue) models. C. we employ well-known evaluation metrics to assess the performance of these models and to compare model-based models with code-based models. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 1
Details of code-based dataset and model-based dataset of Unified bug dataset.

Dataset	No. of samples		No. of defective samples	
	Source	Design	Source	Design
Promise	15 521	15 495	5589	5583
Bugcatcher	2305	2305	586	586
BugPrediction	4882	4880	812	811
Github	27 215	20 467	2379	1907
Zimmerman	25 210	24 259	7662	7275
Unified	47 618	40 842	8780	8301
Total (merged all)	—	67 406	—	16 162

3.1. Dataset construction and object-oriented metrics

The model-based metrics datasets construction from source code datasets is shown in Fig. 2(A). First, we reverse-engineered open source projects to extract a structural design model (class diagram, package diagram) and stored it as META models (XMI). Second, we calculated design metrics using SDmetrics² tool. Finally, we mapped the design metrics with their labels. The dataset constructed as follows:

Source code dataset: We used the Unified bug dataset [17]. The Unified bug dataset is a repository that contains a number of bugs and source code metrics for each class, encompassing a total of 82 open-source Java projects from five public bug datasets Promise, Bugcatcher, BugPrediction, Github and Zimmerman.

Reverse engineering: A class diagram was generated from the source code using the AltovaUML³ tool for each project. We stored the class diagrams in XMI format.

Calculating design metrics: Object-oriented design (OOD) metrics from XMI files were measured using the SDMetrics tool. Here, we have selected 24 metrics that can be measured from class diagrams to assess object-oriented design in terms of size, inheritance, and coupling. The design metrics we used are shown in Table 2.

Matching labels: In this step, the design metrics measured for each class were matched with the bug records in the Unified Bug Dataset and labeled as defective (1) or clean (0). At this stage, we were unable to match the anonymous class. The model-based defect dataset that we created is available online.⁴

Details of code metric-based dataset and newly created model-based dataset of Unified bug dataset shown in Table 1. We would like to note that the Unified Bug Dataset, developed by Ferenc et al. [17], actually merges three datasets: Promise, Bugcatcher, and GitHub. Therefore, we created a comprehensive merged dataset that combines all five datasets, as shown in Table 1. The reason the number of samples in the model-based dataset is smaller than in the code-based dataset is that we were unable to match anonymous classes.

3.2. Model training

To develop best model for our newly created datasets, we considered conventional ML models and the two DL architectures. We split the datasets into 75% for training and 25% for testing. The training

² <https://www.sdmetrics.com/>.

³ <https://www.altova.com/umodel>.

⁴ <https://gitlab.com/design-metric/metric-based-design-defect-dataset>.

Table 2
Design metrics.

	Metrics	Description
Size	NumAttr	The number of attributes in the class
	NumOps	Number of operations in a class
	NumPubOps	Number of public operations in a class
	Setters	Number of operations with a name starting with 'set'
	Getters	Number of operations with a name starting with 'get', 'is', or 'has'
Inheritance	IFImpl	Number of interfaces the class implements
	NOC	Number of subclasses of the class
	NumDesc	Number of descendants of the class
	NumAnc	Number of ancestors of the class
	DIT	Depth of the class in the inheritance tree
	CLD	Class to leaf depth
	OpsInh	Number of inherited operations
	AttInh	Number of inherited attributes
	NumDirClients	Number of elements directly implementing the interface
	NumIndClients	Number of elements implementing a descendent of the interface
Coupling	NumAssEl_ssc	Number of associated elements in the same scope as the class
	NumAssEl_sb	Number of associated elements in the same scope branch as the class
	NumAssEl_nsb	Number of associated elements not in the same scope branch as the class
	EC_Attr	Number of times the class is externally used as attribute type
	IC_Attr	Number of attributes in the class having another class or interface as their type
	EC_Par	Number of times the class is externally used as parameter type
	IC_Par	Number of parameters in the class having another class or interface as their type
	Assoc	Number of elements the interface has an association with
	Nesting	Nesting level of the class

set is used to train the model and identify the best hyperparameters, while the testing set is used to compare the performances of the models. As shown in Fig. 2(B), we first applied preprocessing techniques including standardization-where each metric was reduced by its mean and divided by its standard deviation and resampling-where generating synthetic samples that are similar to the existing minority class samples. As shown in Table 1, our datasets are unbalanced. Hence, we used Synthetic Minority Oversampling Technique (SMOTE) [27] to tackle our imbalanced datasets. SMOTE is a widely used, powerful technique for increasing the number of minority class instances. Here, we would like to note that standardization is applied to the testing set, but resampling is not. We also note that missing value filling and label binarization (converting the number of defects found in a class to “defective” 1 and “clean” 0) are already applied.

In this work, we utilized the various ML algorithms including Linear Regression (LR), k-nearest neighbor (KNN), Decision Tree (DT), Random Forest (RF), XGBoost, and ExtraTree (ET). Additionally, we considered two DL architectures: Shallow MLP with one or two hidden layers and TabTransformer (TabTF) [28] developed by Huang et al. TabTransformer is built upon Transformers and shown effective on tabular data.

For default MLP, we considered MLP with 1 hidden layer. The initial structure of the MLP is shown below:

$$MLP = Input(I) \rightarrow FC(N, Activation) \rightarrow Output(1, Sigmoid) \quad (1)$$

Here, I indicates input features (24 for model-based and 60 for code-based model), FC indicates fully-connected (dense) layer, and N indicates the number of neurons. The default model is configured with 50 neurons using ReLU activation ($N = 50$ and $Activation = \text{ReLU}$), similar to model presented in [11]. The other configuration for this

model is set to batch size of 128, a learning rate of 0.01, and uses the Adam optimizer with binary cross-entropy loss. We will further expand this model by adding two hidden layers, tuning the number of neurons per layer, activation functions, and learning rate using KerasTuner [29], a hyperparameter optimization framework. The hyperparameter tuning process is discussed in Section 4.1.

We selected the best model for each algorithm using fine-tuning hyperparameters with random search. Random search is a well-known technique that provides almost comparable results to grid search in considerably less time.

3.3. Performance comparison

For performance evaluation, we utilized the well-known evaluation measure: accuracy, precision, recall, F-measure (also called F1 score) and AUC. An explanation of each measure with its equation is discussed below:

- **Accuracy (Acc)** is the measure of the correctness of prediction model.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

- **Precision (P)** is a positive predicted value.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

- **Recall (R)** (also called sensitivity) is the percentage of the “defective” classes that are predicted “defective”.

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

- **F-measure (F1)** is the harmonic mean of precision and recall.

$$F - measure = \frac{2 * Precision * Recall}{Precision + Recall} \quad (5)$$

- **Area Under the ROC curve (AUC)** is a combined measure of the sensitivity and specificity, where specificity is the percentage of the “clean” classes that are predicted “clean”. AUC can have a value between [0,1], with 0 representing worst and 1 representing the best classifier.

As shown in Fig. 2(C), to compare model-based and code-based models, we applied the same training process to the code-based metrics datasets. The evaluation result is discussed in Section 4.3.

4. Experiments and results

In this section, we compare and report the results obtained in the experimental work. In the experiments, we implemented LR, KNN, DT, RF, XGBoost, Extra tree (ET) machine learning models using the scikit-learn [30] library, while MLP and TabTF deep learning models are implemented by using the Tensorflow.

4.1. The effect of preprocessing and hyperparameters

Preprocessing

The first step, we compared the results of default algorithms (algorithms trained and evaluated by keeping unchanged their default hyperparameters) on the original data and on the standardized data. The results show in Table 3. Data standardization slightly improved the performance of the models. The average prediction performance of the models improved ranging 0.01% to 0.34%. In some cases such as LR, XGBoost and TabTF, the data standardization does not affect the performances. However, for MLP, it enhances the AUC by 2.54% and F-measure by 3.25%. For further experiments, data standardization will be used to facilitate faster convergence during hyperparameter tuning of the models.

Table 3

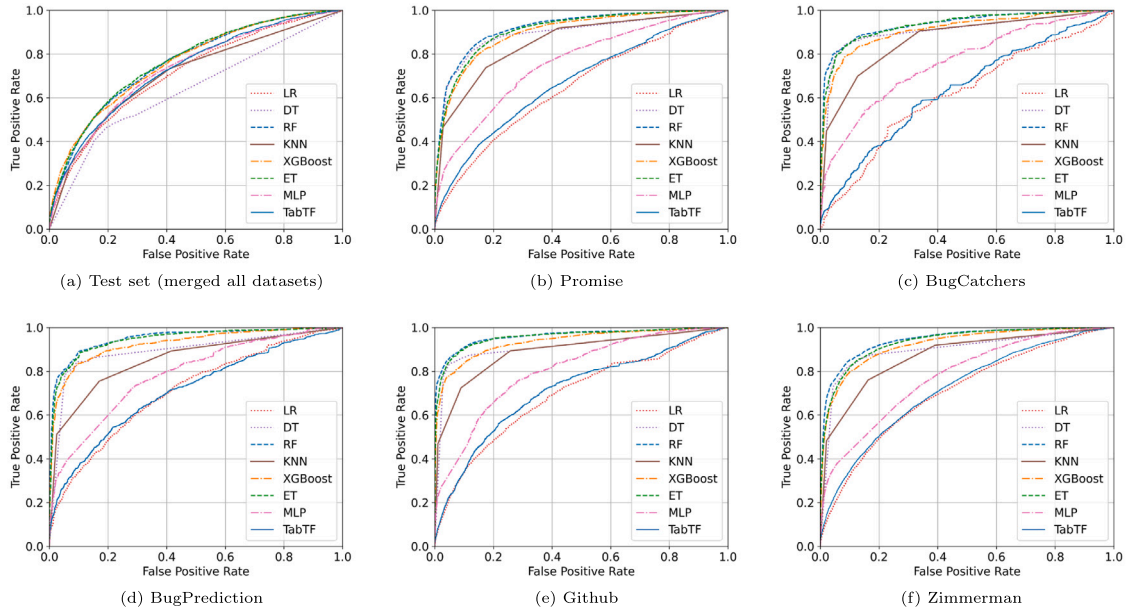
Model performances trained on the original data, standardized data, standardized and resampled data.

Models	None					Standardization					Standardization + Resampling				
	Acc	P	R	F1	AUC	Acc	P	R	F1	AUC	Acc	P	R	F1	AUC
LR	77.29	69.97	55.15	53.95	55.15	77.29	69.97	55.15	53.95	55.15	70.92	63.00	65.41	63.65	65.41
DT	74.77	64.82	64.07	64.41	64.07	74.67	64.67	63.90	64.24	63.90	73.32	63.03	62.72	62.87	62.72
RF	78.55	70.09	64.46	66.07	66.46	78.54	70.08	64.50	66.11	64.50	77.65	68.59	65.57	66.68	65.57
KNN	77.66	68.42	63.64	65.04	63.64	77.08	67.25	62.18	63.50	62.18	72.60	64.26	66.28	64.95	66.28
XGBoost	79.09	72.59	61.48	63.10	61.48	79.09	72.59	61.48	63.10	61.48	78.92	71.09	63.51	65.28	63.51
ET	78.30	69.61	64.14	65.69	64.14	78.21	69.45	63.94	65.49	63.94	77.71	68.70	65.76	66.86	65.76
MLP	77.55	70.58	56.00	55.37	56.00	77.99	70.38	58.54	59.25	58.54	75.59	64.99	62.09	63.03	62.09
TabTF	77.47	70.18	56.36	55.95	56.36	77.90	71.64	56.86	56.69	56.86	69.11	62.69	66.04	63.10	66.04
Average	77.59	69.53	60.66	61.20	60.66	77.60	69.50	60.82	61.54	60.82	74.48	65.79	64.67	64.55	64.67

Table 4

Best models for each algorithm: Hyperparameters and Performance metrics.

Models	Acc	P	R	F1	AUC	Best parameters
LR	70.92	63.00	65.41	63.65	65.41	solver: lbfgs, penalty: L2, C: 1
DT	73.58	63.31	62.88	63.08	62.88	min_samples_leaf: 1, max_features: None, max_depth: 64, criterion: entropy
RF	77.94	69.04	65.85	67.02	65.85	n_estimators: 500, max_features: log2, max_depth: 32
KNN	73.11	64.57	66.32	65.22	66.32	n_neighbors: 3
XGBoost	78.70	70.30	65.81	67.29	65.81	subsample: 0.6, n_estimators: 100, min_child_weight: 1, max_depth: 20, max_delta_step: 5, learning_rate: 0.1, gamma: 0, colsample_bytree: 0.9
ET	77.84	68.91	65.99	67.09	65.99	n_estimators: 500, min_samples_split: 8, min_samples_leaf: 2, max_features: None, max_depth: 100, criterion: entropy
MLP	74.31	64.73	64.82	64.77	64.82	hidden_layers: 2, layer1: (neurons: 140, activation: Tanh) layer2: (neurons: 80, activation: ReLU), learning_rate: 0.01
TabTF	67.48	62.63	66.63	62.50	66.63	learning_rate: 0.0005, initialization: uniform
Average	74.28	65.81	65.46	65.08	65.46	

**Fig. 3.** ROC curves for comparison of ML and DL models on a specific dataset.

As mentioned in Section 3, we applied resampling technique to handle imbalanced dataset by oversampling the minority class. We used the SMOTE technique implemented in the Scikit-learn library. The results in Table 3 show that class balance can significantly affect model performance. The average predictions of the models improved to 3.85%, 3.01% and 3.85% in terms of recall, F-measure and AUC, respectively, after resampling. However, accuracy and precision decreased. Moreover, the AUC and F-measure of LR improved by around 10%. The AUC of XGBoost and KNN improved by 2.03% and 4.1%, while the F-measure increased by 2.18% and 1.45%, respectively. Additionally, the AUC of MLP and TabTF improved by 3.55% and 9.18%, while the F-measure increased by 3.79% and 6.41%, respectively. Although, in some cases, such as with DT, AUC and F-measure decreased after oversampling. We will discuss this in Section 4.2.

Hyperparameter tuning

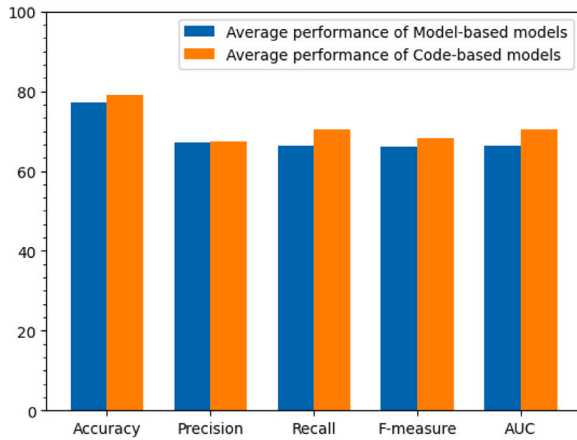
In the previous experiment, the considered ML and DL models were trained and evaluated by their default hyperparameters. For all models, we conducted hyperparameter tuning using random search. The hyperparameters of ML algorithms were tuned using 10-fold cross-validation on the training set, utilizing RandomizedSearchCV from scikit-learn. The hyperparameters of DL algorithms were adjusted on validation set (25% of training set), utilizing RandomSearch from KerasTuner.

We defined the search space for the RF with the number of trees ranging from 100 to 500 (increment by 100), a maximum depth varying from 2 to 128 with exponential growth of 2, and maximum features set to 'log2', 'sqrt', and None. For LR, we defined the search space with the $C \in \{1, 10, 100, 1000\}$, solver set to 'newton-cg', 'lbfgs' and

Table 5

Model-based vs. Code-based: Fine-tuned model performances on the Unified bug dataset.

Algorithms	Model-based model (24 features)					Code-based model (60 features)				
	Acc	P	R	F1	AUC	Acc	P	R	F1	AUC
LR	71.23	61.77	65.46	62.44	65.46	77.87	64.87	67.50	65.88	67.50
DT	77.15	65.24	65.61	65.42	65.61	76.68	64.23	67.85	65.41	67.85
RF	81.59	71.48	67.17	68.78	67.17	82.25	70.36	71.29	70.80	71.29
KNN	75.93	65.30	68.03	66.27	68.03	75.28	65.83	73.17	67.07	73.17
XGBoost	81.68	71.66	66.92	68.64	66.92	82.76	70.95	70.75	70.85	70.75
ET	81.59	71.49	68.12	69.47	68.12	82.62	70.83	71.22	71.02	71.22
MLP	79.65	67.61	63.47	64.87	63.47	78.61	66.37	69.92	67.66	69.92
TabTF	69.55	62.12	67.06	62.34	67.06	77.26	66.41	72.26	67.95	72.26
Average	77.30	67.08	66.48	66.03	66.48	79.17	67.48	70.49	68.33	70.49

**Fig. 4.** Comparison on average performance of models trained on model-based metrics and code-based metrics.

‘liblinear’. For XGboost, we defined the search space similarly to the RF with max depth from 2 to 24, minimum child weight from 1 to 9 (increment by 2), subsample ratio ranging from 0.5 to 1.0 by 0.1, subsampling of columns ranging 0.6 to 1.0 by 0.1, learning rate set to {0.01, 0.1, 0.2} and gamma ranging from 0 to 0.5 by 0.1. For DT, we defined the search space with the max depth ranging from 2 to 128 with exponential growth of 2, minimum number of samples set to {0.1, 1, 3, 5, 10}, criterion set to ‘entropy’, ‘log_loss’ and ‘gini’. For KNN, we considered K from 1 to 32. The search space for the Extra Trees is configured similarly to RF.

For the MLP, we considered configurations with 1 and 2 hidden layers, varying from 10 to 200 neurons in increments, using different activation functions (‘relu’, ‘sigmoid’, ‘softplus’, ‘softsign’, ‘tanh’, ‘selu’, ‘elu’ and ‘exponential’), and including or excluding a dropout layer. For tuner, we set 100 trials and each model was trained 100 epoch with different learning rate varying from 0.0001 to 0.1 with Adam optimizer, batch size of 128, and the accuracy as the loss function. We also implemented early stopping with a patience of 5, meaning training would stopped if there was no improvement in the training loss for 5 consecutive epochs. For the TabTF, we considered learning rate varying from 0.0001 to 0.1, initialization set to ‘kaiming’, ‘normal’ and ‘uniform’, layers set to ‘32-16’, ‘64-32’ and ‘128-64’, and batch size varying 128 to 512.

The best hyperparameters for each model are shown in Table 4, along with the performance metrics. All models achieved higher performance after hyperparameter tuning. The average predictions of the models improved by 0.79% in terms of recall and AUC, respectively, after tuning. Moreover, F-measure and AUC of XGBoost increased by 2.01% and 2.20%, while F-measure and AUC of MLP improved by 1.74% and 2.73%, respectively.

4.2. Performance of model-based defect prediction

In this subsection, we compare the performances of fine-tuned ML and DL models on model-based defect prediction. As shown in Table 4, tree-based models such as RF, XGBoost and ET performs best among the ML models, while TabTF achieves nearly identical results in terms of AUC. To compare the performances of the models, we plotted the ROC curve using the specific dataset for each model. Fig. 3 shows the ROC curves of ML and DL models in a comparative manner. Fig. 3 consists of six subfigures: Fig. 3(a) presents ROC curve on test set (25%) of merged all datasets, while the remaining 5 focus on specific datasets without splitting train and test.

It can be observed from the ROCs that RF model slightly outperforms all the classifiers over most datasets. Interestingly, the ROC of DT performs better than LR, MLP and KNN comparing to the results shown in Tables 3, 4. It is possible that DT offer better generalization ability (generalizability) compared to LR, MLP and KNN. Conversely, the ROC for TabTF performs worse on specific datasets, suggesting that TabTF may be overfitting.

4.3. Code-based model vs. Model-based model

In this subsection, we compare the performances of code-based models and model-based models. In the previous experiments, we used comprehensive datasets containing 67,406 samples from five datasets. For this experiment, to ensure a fair comparison, we are using a unified dataset that includes 40,842 samples from three different datasets, which is the same as the Unified Bug Dataset. Table 5 shows the results of the ML and DL models trained on the model-based metrics and code-based metrics. It can be seen that ET model outperforms other models in terms of F-measure for both cases. On top of that, we plot the averages of accuracy, precision, recall, F-measure, and AUC for the models trained on code metrics and model metrics, as shown in Fig. 4. It clearly shows that the performances of the model trained on model metrics are almost identical to the model trained on code metrics. We can conclude that our approach can effectively identify defects at the design stage, prior to the implementation stage of the SDLC.

4.4. Generalizability evaluation

In this experiment, we conducted a cross-dataset evaluation to determine to the fine-tuned model on one dataset may be utilized to detect defect in other (unseen) datasets. To achieve this, we first fine-tune the model using a specific dataset (e.g., Promise), and then we evaluate the model’s performance on the remaining datasets (i.e., BugCatchers, BugPrediction, Github and Zimmerman). With this experiment we want to examine the generalization ability of our approach, i.e., to which extent a model trained on a specific dataset can be used to detect defects in new unseen dataset. We used RF model for this experiment due to the results shown in Fig. 3. In Fig. 5, we plotted the result of the cross-dataset evaluation. It can be seen that the RF model demonstrates better accuracy while performing moderately on other metrics. However, it features smaller boxes and shorter whiskers (indicating less dispersion)

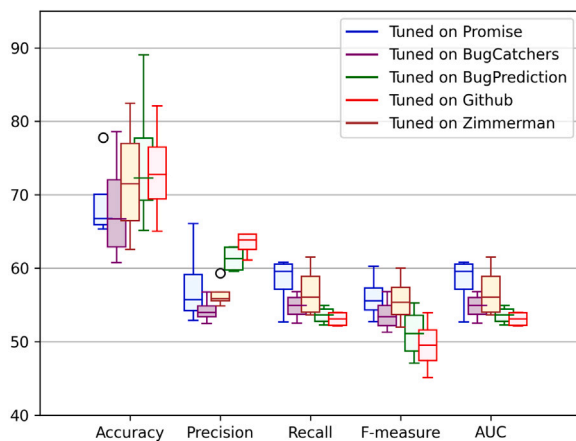


Fig. 5. Cross-dataset evaluation.

regardless of whether it is trained on a large (e.g., Zimmerman, Github) or small (e.g., Bugcatchers, BugPrediction) dataset. Furthermore, based on the results reported in [12] and the experiments we conducted, we believe that our approach might already be useful for defect prediction from class diagram at design phrase of the SDLC.

5. Threads to validity

This section covers the potential risks and how we managed them.

Conclusion validity: The possible threads of our study are accuracy of dataset and effectiveness of metrics selected. This study uses the unified bug dataset, and the authors believe that data are collected and processed in a consistent manner. However, we acknowledge that the dataset contains several versions of typical projects, which may lead to dataset overlap. We employed several measures commonly used in the literature, particularly for imbalanced datasets, including accuracy, precision, recall, F-measure, and AUC.

Construct validity: Our model-based datasets were created using reverse-engineering, thus we identified potential problems in the dataset construction. To generate class diagrams from source codes, we used AltovaUML which is powerful and popular tool for software engineering. To reduce label mapping errors, we automated label matching using the class's full path. Furthermore, we identified potential problems in the adopted methodology. The implementation of ML models and DL models is considered thread. To mitigate this, we used the reliable library from scikit-learn and Tensorflow. Moreover, we constructed shallow DL model, the results may come out to be different with different parameters such as adding more hidden layers. Additionally, better results can be obtained if we apply more ML and DL techniques.

Internal validity: An internal concern of our study is that we concentrate on static design metrics that obtained from class diagram. We believe that these metrics adequately represent the characteristics of the class, and we obtained them using the SDmetrics tool, which is commonly used to calculate object-oriented design metrics. Dynamic design metrics can be calculated from interaction diagrams, which will be investigated in further studies.

External validity: External validity associated with the ability to generalize the results beyond the experiment. For this reason, we conducted cross-dataset experiments to evaluate the generalizability of the proposed method. The results show that our method can be generalized to other systems, but with relatively low performance. In this study, we constructed class diagrams from Java projects; however, the static design model is independent of programming language. Therefore, our findings will be applicable to projects that will be implemented in any programming language.

6. Conclusion

In this study, we presented the novel approach to predict defects in class diagram using ML and DL models. We used software metrics because they provide a quantifiable way to analyze the software, making it easier to identify defects. Specifically, we first created the model-based metrics dataset using reverse engineering from a code metric-based dataset. Second, we adapted ML and DL models and carefully tuned their hyperparameters. Lastly, we evaluated the performance of ML and DL models and demonstrated that they are effective methods compared to code-based models. Additionally, we conducted cross-dataset evaluations, and the experimental results demonstrate that our approach can detect defects at the model level on unseen datasets.

In future studies, we plan to expand our dataset including sequence diagrams and use-case diagrams, and train DL models. In the meantime, we consider our findings a successful step toward detecting defects in the design phase of the SDLC, and we are continuing our research on identifying defects in the early stages of the SDLC, specifically during the requirements specification phase.

CRediT authorship contribution statement

Batnyam Battulga: Writing – original draft, Conceptualization, Writing – review & editing. **Lkhamrolom Tsoodol:** Writing – review & editing, Data curation. **Enkhzol Dovdon:** Writing – review & editing, Investigation. **Naranchimeg Bold:** Writing – review & editing, Supervision, Conceptualization. **Oyun-Erdene Namsrai:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National University of Mongolia under Grant P2020-3971.

Data availability

I have shared link to our data at <https://gitlab.com/design-metric/metric-based-design-defect-dataset>.

References

- [1] Li Z, Jing X-Y, Zhu X. Progress on approaches to software defect prediction. *Iet Softw* 2018;12(3):161–75.
- [2] Wahono RS. A systematic literature review of software defect prediction. *J Softw Eng* 2015;1(1):1–16.
- [3] Vitharana P. Defect propagation at the project-level: results and a post-hoc analysis on inspection efficiency. *Empir Softw Eng* 2017;22(1):57–79.
- [4] Thota MK, Shajin FH, Rajesh P, et al. Survey on software defect prediction techniques. *Int J Appl Sci Eng* 2020;17(4):331–44.
- [5] Stradowski S, Madeyski L. Industrial applications of software defect prediction using machine learning: A business-driven systematic literature review. *Inf Softw Technol* 2023;159:107192.
- [6] Pachouly J, Ahirrao S, Kotecha K, Selvachandran G, Abraham A. A systematic literature review on software defect prediction using artificial intelligence: Datasets, data validation methods, approaches, and tools. *Eng Appl Artif Intell* 2022;111:104773.
- [7] Arcelli Fontana F, Mäntylä MV, Zanoni M, Marino A. Comparing and experimenting machine learning techniques for code smell detection. *Empir Softw Eng* 2016;21:1143–91.
- [8] Akimova EN, Bersenev AY, Deikov AA, Kobylkin KS, Konygin AV, Mezentshev IP, Misilov VE. A survey on software defect prediction using deep learning. *Math* 2021;9(11):1180.

- [9] Giray G, Bennin KE, Köksal Ö, Babur Ö, Tekinerdogan B. On the use of deep learning in software defect prediction. *J Syst Softw* 2023;195:111537.
- [10] Radjenović D, Heričko M, Torkar R, Živković A. Software fault prediction metrics: A systematic literature review. *Inf Softw Technol* 2013;55(8):1397–418.
- [11] Goyal S. Static code metrics-based deep learning architecture for software fault prediction. *Soft Comput* 2022;26(24):13765–97.
- [12] Ferenc R, Bán D, Grósz T, Gyimóthy T. Deep learning in static, metric-based bug prediction. *Array* 2020;6:100021.
- [13] Wang S, Liu T, Tan L. Automatically learning semantic features for defect prediction. In: *Proceedings of the 38th international conference on software engineering*. 2016, p. 297–308.
- [14] Dam HK, Tran T, Pham T, Ng SW, Grundy J, Ghose A. Automatic feature learning for vulnerability prediction. 2017, arXiv preprint [arXiv:1708.02368](https://arxiv.org/abs/1708.02368).
- [15] Shi K, Lu Y, Chang J, Wei Z. PathPair2Vec: An AST path pair-based code representation method for defect prediction. *J Comput Lang* 2020;59:100979.
- [16] Zakeri-Nasrabadi M, Parsa S, Esmaili E, Palomba F. A systematic literature review on the code smells datasets and validation mechanisms. *ACM Comput Surv* 2023;55(13s):1–48.
- [17] Ferenc R, Tóth Z, Ladányi G, Siket I, Gyimóthy T. A public unified bug dataset for java. In: *Proceedings of the 14th international conference on predictive models and data analytics in software engineering*. 2018, p. 12–21.
- [18] Maddeh M, Ayouni S, Alyahya S, Hajjej F. Decision tree-based design defects detection. *IEEE Access* 2021;9:71606–14.
- [19] Alazba A, Aljamaan H, Alshayeb M. Automated detection of class diagram smells using self-supervised learning. *Autom Softw Eng* 2024;31(1):29.
- [20] Fioravanti F, Nesi P. A study on fault-proneness detection of object-oriented systems. In: *Proceedings fifth European conference on software maintenance and reengineering*. IEEE; 2001, p. 121–30.
- [21] Briand LC, Wüst J, Daly JW, Porter DV. Exploring the relationships between design measures and software quality in object-oriented systems. *J Syst Softw* 2000;51(3):245–73.
- [22] Aggarwal K, Singh Y, Kaur A, Malhotra R. Investigating effect of design metrics on fault proneness in object-oriented systems.. *J Object Technol* 2007;6(10):127–41.
- [23] Gaur S, Kaur S, Kaur I. Validation of software quality models using machine learning: An empirical study. *Int J Comput Appl* 975:8887.
- [24] Malhotra R, Jain A. Fault prediction using statistical and machine learning methods for improving software quality. *J Inf Process Syst* 2012;8(2):241–62.
- [25] Oyun-Erdene N LT. Predicting class fault proneness using data mining classification techniques. *J Adv Res Dyn Control Syst* 2018;10:585–90.
- [26] Lin T, Wang Y, Liu X, Qiu X. A survey of transformers. *AI Open* 2022;3:111–32.
- [27] Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP. SMOTE: synthetic minority over-sampling technique. *J Artificial Intelligence Res* 2002;16:321–57.
- [28] Huang X, Khetan A, Cvitkovic M, Karnin Z. Tabtransformer: Tabular data modeling using contextual embeddings. 2020, arXiv preprint [arXiv:2012.06678](https://arxiv.org/abs/2012.06678).
- [29] O'Malley T, Bursztein E, Long J, Chollet F, Jin H, Invernizzi L, et al. KerasTuner. 2019, <https://github.com/keras-team/keras-tuner>.
- [30] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E. Scikit-learn: Machine learning in python. *J Mach Learn Res* 2011;12:2825–30.